

```

10.     idx = 0
11.     for length in rail_lengths:
12.         rails.append(ciphertext[idx:idx + length])
13.         idx += length
14.     plaintext = ''
15.     for i in range(len(ciphertext)):
16.         rail_idx = i % num_rails
17.         plaintext += rails[rail_idx][0]
18.         rails[rail_idx] = rails[rail_idx][1:]
19.     return plaintext

```

Now, let us go through the code line by line to understand how it works. Line 7 defines the `rail_fence_decrypt` function. It takes two arguments: `ciphertext`, which contains the encrypted message and `num_rails`, which specifies the number of rows (rails) used during encryption. Line 8 determines how many characters belong to each rail. Line 9 creates an empty list `rails` to store substrings of `ciphertext` corresponding to each rail. Line 10 sets the index pointer `idx` to 0. This will track the current position in the `ciphertext` while splitting it into rails. Line 11 loops through each rail's length in `rail_lengths`. Line 12 extracts a substring from the `ciphertext` starting at `idx` with a length equal to `length` and appends the substring to the list `rails`. Line 13 moves the index pointer `idx` forward by the length of the current rail, preparing for the next slice. Line 14 creates an empty string `plaintext` to hold the reconstructed message. Line 15 iterates through the ciphertext, one character at a time, based on its length. Line 16 calculates the index of the rail (`rail_idx`) from which the next character should be taken. The modulus operator (%) is used to cycle through the rails. Line 17 appends the first character ([0]) from the current rail (`rails[rail_idx]`) to `plaintext`. Line 18 removes the first character from the current rail after it is added to `plaintext`. Line 19 returns the fully reconstructed plaintext as the output of the function.

The code below performs the testing process in the program `railfence.sage` (line numbers are included for clarity):

```

20. plaintext = "OPENSOURCEFORYOU"
21. num_rails = 3
22. ciphertext = rail_fence_encrypt(plaintext, num_rails)
23. decrypted_text = rail_fence_decrypt(ciphertext,
    num_rails)
24. print("Plaintext:", plaintext)
25. print("Ciphertext:", ciphertext)
26. print("Decrypted Text:", decrypted_text)

```

Now, let us try to understand how the testing works. Line 20 assigns the plaintext message 'OPENSOURCEFORYOU' to the variable `plaintext`. This message will be encrypted using the rail fence cipher. Line 21 sets the number of rails (rows)

to 3. This determines how the plaintext will be arranged during the encryption process. Line 22 calls the function `rail_fence_encrypt`, passing `plaintext` and `num_rails` as arguments. The function encrypts the plaintext using the Rail Fence cipher and assigns the resulting ciphertext to the variable `ciphertext`. Line 23 calls the function `rail_fence_decrypt`, passing `ciphertext` and `num_rails` as arguments. The function decrypts the ciphertext using the Rail Fence cipher and assigns the resulting plaintext to the variable `decrypted_text`. Lines 24 to 26 display the plaintext, its encrypted ciphertext, and the decrypted ciphertext, confirming the successful encryption and decryption process.

The SageMath program `railfence.sage` can be executed by running the command `sage railfence.sage` in a terminal. Figure 2 shows the execution and output of the SageMath program `railfence.sage`. To verify the program's functionality, replace Line 20 with the following line of code: `plaintext = "CONSIDERABLE"`. Executing this modified program should yield the ciphertext 'CSEBOIRLNDAAE', as expected from our earlier example. Next, in the original code, replace Line 21 with the following line of code: `num_rails = 4`. Running this modified program will produce the ciphertext 'OSCRPOEYEU FONROU', confirming that the encryption result varies with the number of rails. With this discussion, we have covered some key classical encryption techniques. It is now time to advance to modern cryptographic methods.

```

# sage railfence.sage
Plaintext: OPENSOURCEFORYOU
Ciphertext: ONUERUPSRFYEOCOO
Decrypted Text: OPENSOURCEFORYOU

```

Figure 2: Execution of the SageMath program `railfence.sage`

Introduction to symmetric-key cryptography

Symmetric-key cryptography is a cornerstone of modern cryptography, providing efficient and secure methods for protecting data in applications such as secure communications, financial transactions, and data storage. This type of encryption employs a single key for both encrypting and decrypting data. The security of this method hinges on the secrecy of the key, which must be shared confidentially between the sender and receiver.

Common symmetric-key algorithms include the Advanced Encryption Standard (AES) and the Data Encryption Standard (DES). While these algorithms are efficient and fast, the primary challenge in symmetric-key cryptography lies in the secure distribution and management of the secret key between the involved parties.

To implement many of the cryptographic functions discussed in this series, we will need to install PyCryptodome, a Python package that provides implementations for a variety of cryptographic algorithms. SageMath, a powerful